Optimizing HDFS Reliability & Performance via the SI-CL-SDEO Algorithm

Based on: D. Dhinakaran et al., "SI-CL-SDEO algorithm for improving HDFS performance," *Results Eng.*, vol. 26, p. 104773, 2025.

Shivam (123AD0028)
Dinesh Sharma (123AD0013)
Rehant Roy (123AD0010)

Table of Contents

Problem Statement

Introduction

Literature Survey

Methodology

Results & Discussion

Conclusion

Future Work

References

Problem Statement

Core Challenge

Efficient data block replication in large-scale HDFS remains unsolved — no single strategy jointly optimizes all critical metrics.

- **Static Replication:** Fixed RF does not adapt to workload changes or node failures.
- **Availability vs. Overhead:** Higher RF boosts fault tolerance but increases congestion and latency.
- **NP-Hard Placement:** Optimal DataNode selection is intractable at scale.

Existing Methods Fall Short:

- **HSHCD:** Static; zero real-time adaptability.
- **MBFOA:** Slow convergence; $O(n^2 T)$.
- **HWACOA:** High overhead; $O(nT^2)$.
- **LW-PSO:** Premature convergence; $O(n^2 T)$.

Research Goal

Design **SI-CL-SDEO** — a hybrid metaheuristic that dynamically optimizes HDFS replication with **linear** $O(n \cdot T)$ complexity.

HDFS Architecture & The Replication Bottleneck

HDFS Master-Slave Topology:

- **NameNode:** Manages namespace, maps blocks to DataNodes, stores metadata in memory.
- **Secondary NameNode:** Merges namespace image/edit logs — *not* a failover node.
- **DataNodes:** Store data blocks; send heartbeats & block reports to NameNode.

Default Replication (RF = 3):

- 2 replicas within the same rack.
- 1 replica on a different rack for rack-failure tolerance.

The Core Trade-off

High RF: ↑ Availability, ↑ Fault tolerance
↓ Bandwidth, ↑ Write latency

Low RF: ↓ Storage overhead
↑ Risk of data loss

Communication Model

- Client ↔ NameNode: metadata ops.
- Client ↔ DataNode: data read/write.
- DataNode → NameNode: heartbeats.

Introducing SI-CL-SDEO

SI-CL-SDEO: Swarm Intelligence – Chaotic Leader & Salp Differential Evolution Optimization

Hybrid metaheuristic combining **SSO** (exploration) + **DE** (exploitation) to dynamically optimize HDFS replication.

Salp Swarm Optimization (SSO)

Leader-follower chain. Provides **global exploration** — avoids local optima by broadly searching the solution space.

Differential Evolution (DE)

Mutation & crossover on vector differences. Provides **local exploitation** — refines promising solutions rapidly.

Transition Factor smoothly shifts from SSO to DE:

$$x_{hyb}^{t+1} = \left(1 - \frac{t}{T}\right) x_{SSO}^{t+1} + \frac{t}{T} x_{DE}^{t+1}$$

Performance Highlights:

- **40%** ↑ data availability (high RF)
- **20%** ↓ execution time
- **96.98%** network traffic efficiency
- Complexity: $\mathcal{O}(n \cdot T)$ — linear

Existing HDFS Optimization Methods

Method	Approach	Limitation	Complexity
HSHCD	Hot/cold data tiering via sorting.	Static; cannot adapt in real-time.	$O(N \log N)$
MBFOA	Bacterial foraging for data replication.	Slow convergence; local optima.	$O(n^2 T)$
HWACOA	Weighted Ant Colony for scheduling.	High pheromone-update overhead.	$O(nT^2)$
LW-PSO	Lateral Wolf Particle Swarm.	Premature convergence; low diversity.	$O(n^2 T)$
KM-MBFO	MapReduce + bacterial foraging.	Ignores fault tolerance & availability.	$O(n^2 T)$

Other methods (EWSO, HDFS-AIO, HPMR, ACO-GA) address narrow sub-problems — small file storage, I/O patterns, or query optimization — but lack **holistic HDFS optimization**.

Research Gap & Motivation

Flaws in Existing Methods

- **MBFOA & LW-PSO:** $O(n^2 T)$ — prohibitive at petabyte scale.
- **HWACOA:** $O(nT^2)$ — bottlenecks in long-running jobs.
- **HSHCD:** Lightweight but entirely static.
- All methods optimize only *1–2* metrics at a time.

Recent Hybrids (Not HDFS-Specific)

CCSA, RCOA, ANFIS-BWO, MOWOS improve cloud scheduling but do *not* address HDFS data replication.

The Research Gap

A **scalable, linear-time** hybrid that simultaneously optimizes:

- Data availability
- Execution time
- Fault tolerance
- Network utilization
- Read/write latency

. . . without quadratic bottlenecks.

Solution: SI-CL-SDEO achieves $\mathcal{O}(n \cdot T)$ via SSO's linear position updates combined with DE's lightweight operators.

HDFS Background: Components & Replication

HDFS Components:

- **NameNode:** Namespace manager; metadata in memory; periodic checkpointing.
- **DataNodes:** Execute read/write; send heartbeats and block reports.
- **Replication (RF=3):** 1 off-rack + 2 same-rack replicas by default.

Replication Factor (ψ) is the Key Variable:

- $\psi = 2$: Basic redundancy, single-node protection.
- $\psi = 3$: Industry standard (fault tolerance + efficiency).
- $\psi = 4$: Critical environments, higher overhead.

SI-CL-SDEO's Role

Dynamically determines **optimal RF** and **DataNode placement** per block by:

- Evaluating latency $L(i)$ and availability $A(i)$.
- Adapting to real-time cluster load.
- Balancing fault tolerance vs. storage cost.

Why Static Strategies Fail

HDFS default RF is fixed at deployment. Under failures or workload spikes, static RF leads to under-replication or resource waste.

SI-CL-SDEO: Framework Overview

Initialization

Input: HDFS data blocks **Params:** n, S, P_{init}, b, C_{end}, C_{init}, F, CR, W_1, W_2
Salps assigned random positions in N-dimensional space (each dimension = a candidate DataNode). Leader selected by fitness; followers initialized.

Phase 1: SSO

- Update leader & follower positions.
- Evaluate fitness per salp.
- $C(t)$ decays to narrow search.

Phase 2: DE

- Mutant vector via vector differences.
- Trial vector via crossover.
- Greedy selection keeps fitter solutions.

Hybrid & Output

$$x_{hyb}^{t+1} = \left(1 - \frac{t}{T}\right) x_{SSO}^{t+1} + \frac{t}{T} x_{DE}^{t+1}$$

Optimized distribution stored in HDFS; NameNode metadata updated.

Mathematical Formulation: SSO Phase

Position Update Equations

Leader Salp:

$$x_{leader}^{(t+1)} = x_{leader}^{(t)} + C(t) \cdot \mathrm{rand}()$$

Follower Salps $(i = 2, \ldots, n)$:

$$x_i^{(t+1)} = \frac{x_i^{(t)} + x_{i-1}^{(t)}}{2}$$

Disturbance Weight:

$$C(t) = C_{init} + (C_{end} - C_{init})\left(1 - \frac{t}{S}\right)^b$$

Fitness Function

$$\mathrm{Fitness}(i) = W_1 \cdot L(i) + W_2 \cdot A(i)$$

- $L(i)$: Network latency (read/write)
- $A(i)$: Data availability
- $W_1 = 0.7$ $(\alpha = 7$, latency priority)
- $W_2 = 0.3$ $(\beta = 3$, availability)
- Weights: $W_k = \frac{\alpha/\beta}{\alpha+\beta}$, adaptable.

Leader is replaced whenever a follower attains higher fitness.

Mathematical Formulation: DE Phase

Three-Step DE Refinement

1. Mutation:

$$v_i = x_{r1} + F \cdot (x_{r2} - x_{r3})$$

$r1, r2, r3$ distinct; F = mutation factor.

2. Crossover:

$$u_{ij} = \begin{cases} v_{ij} & \text{if } \mathrm{rand}_j \leq CR \\ x_{ij} & \text{otherwise} \end{cases}$$

3. Selection:

$$x_i^{(t+1)} = \begin{cases} u_i & \text{if } f(u_i) > f(x_i) \\ x_i & \text{otherwise} \end{cases}$$

Notation Summary

x_i	Target vector
v_i	Mutant vector
u_i	Trial vector
F	Mutation factor
CR	Crossover rate
L, A	Latency, Availability
W_1, W_2	Fitness weights
n, S	Population size, Max iterations
C_{init}, C_{end}	Disturbance weight bounds
b	Decay parameter

Greedy selection guarantees monotonically non-decreasing population quality.

Convergence & Computational Complexity

Complexity: $\mathcal{O}(n \cdot T)$

- **SSO:** $O(n)$ per iteration — linear position updates.
- **DE:** $O(n)$ per iteration — lightweight mutation/crossover.
- **Total:** $O(n \cdot T)$ — linear in both n and T.

Algorithm	Complexity	High-Load ET
SI-CL-SDEO	$O(n \cdot T)$	**204 ms**
HSHCD	$O(N \log N)$	217 ms
MBFOA	$O(n^2 T)$	238 ms
HWACOA	$O(nT^2)$	246 ms
LW-PSO	$O(n^2 T)$	259 ms

Table: Complexity & measured ET (n=50, T=100).

Convergence Guarantees:

- $C(t)$ decays monotonically — search narrows progressively.
- Greedy selection prevents fitness regression.
- Recalculation avoids premature stagnation.

Key Advantage

SI-CL-SDEO is the **only linear-time** method — enables scalability to large distributed clusters.

HDFS Write Pipeline: Setup & Block Writing

Pipeline Setup (Block B)

1. Client → **NameNode**: write request.
2. NameNode returns IPs of DN2, DN5, DN7 (load-balanced, rack-aware).
3. Client pings DNs for readiness.
4. Pipeline established: Client → DN2 → DN5 → DN7.

Block Write (Block C)

1. NameNode selects DN1, DN4, DN6 by proximity & load.
2. Client streams: DN1 → DN4 → DN6.
3. **Recovery:** DN4 fails ⇒ NameNode substitutes DN5; pipeline reconfigured automatically.

ACK Mechanism (Block D)

- DN1 writes → DN3 → ACKs client.
- DN3 writes → DN5 → ACKs DN1.
- DN5 writes; reverse ACK cascade to client.
- Each DN confirms receipt before next packet.
- Failure: NameNode re-routes pipeline transparently.

Final Confirmation

After all ACKs received, NameNode updates metadata: Block D successfully placed across DN1, DN3, DN5. Write operation complete.

Simulation Setup & Evaluation Metrics

Simulation Environment:

- **Hardware:** 16 GB RAM, quad-core 3.4 GHz, 1 TB SSD/node.
- **Software:** Hadoop 3.3.1, Apache JMeter, custom scripts.
- **DataNodes:** 10, 50, 100 **RF:** $\psi = 2, 3, 4$
- **Block sizes:** 64/128/256 MB **Load:** 6000 reqs.
- **Network:** 1/10 Gbps; 10/50 ms latency.
- **Failure rates:** 1%, 5%, 10%
- **Data volumes:** 100 GB / 500 GB / 1 TB

Evaluation Metrics:

- $\text{ARL} = \dfrac{\sum \text{ReadTime}(i)}{N}$
- $\text{AWL} = \dfrac{\sum \text{WriteTime}(i)}{M}$
- $\text{DA} = \dfrac{\text{Uptime}}{\text{Uptime} + \text{Downtime}} \times 100$
- $\text{ET} = \text{End Time} - \text{Start Time}$
- Also: **Fault Tolerance**, **PDR**, **Network Utilization**

SI-CL-SDEO: Complete Algorithm Flow

Initialization

1. Assign random positions to salps in search space.
2. Initialize DE population from salp positions.
3. Select leader salp by fitness criterion (Eq. 4).
4. Compute follower salp positions.

SSO Phase (each iteration)

5. Update leader & follower positions (Eqs. 1, 2).
6. Evaluate fitness per salp (Eq. 4).
7. Update disturbance weight $C(t)$ (Eq. 3).
8. Replace leader if a follower has higher fitness.

DE Phase (each iteration)

9. Generate mutant v_i per target vector (Eq. 5).
10. Generate trial vector u_i via crossover (Eq. 6).
11. Greedy selection: keep u_i if $f(u_i) > f(x_i)$.
12. Periodically recalculate leader salp position.

Hybrid Combination & Output

13. Compute x_{hybrid}^{t+1} using Transition Factor (Eq. 7).
14. Repeat until max iterations S reached.

Output: Optimized block distribution returned; block locations & metadata updated in NameNode.

SI-CL-SDEO: Workflow Diagram

Workflow of the SI-CL-SDEO Algorithm for HDFS Optimization

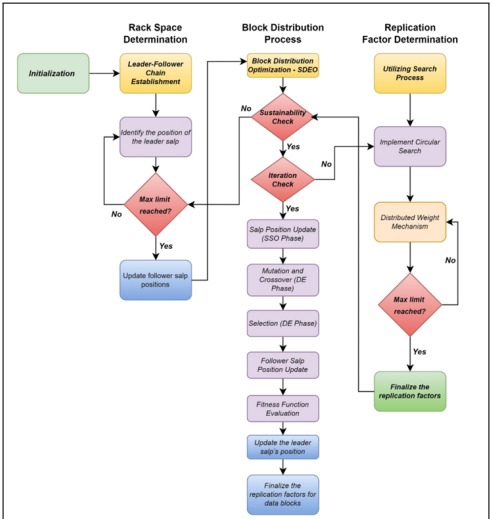
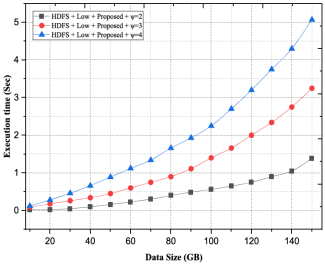
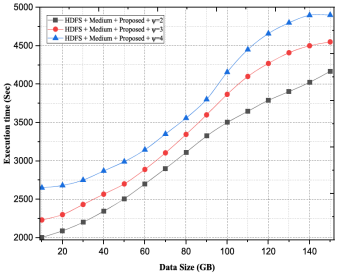


Fig. 2. Workflow of the SI-CL-SDEO Algorithm for HDFS Optimization

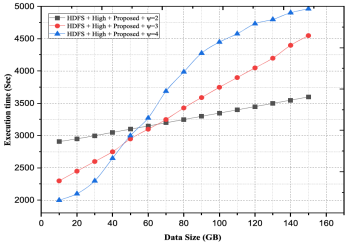
Result 1: Execution Time Under Load Conditions



(a) Low Load ET doubles ψ=2→3 and again ψ=3→4 at 50-unit datasets.

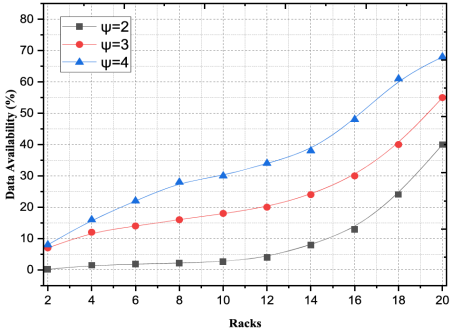


(b) Medium Load ET ↑ ∼13.7% (ψ=2→3) and ∼9.6% (ψ=3→4) for 100-unit datasets.



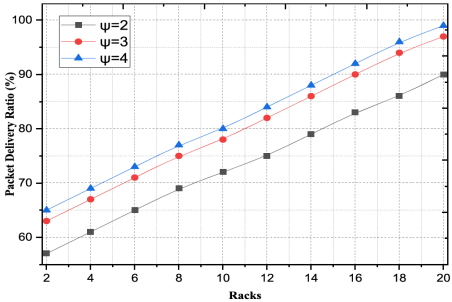
(c) High Load ψ=3 slightly *lower* than ψ=2 at 50 units — better load distribution at higher RF.

Result 2: Data Availability & PDR



(a) Data Availability

(a) Data Availability vs. Racks



(b) Packet Delivery Ratio vs. Racks

Key Values (10 racks)

$\psi = 2 \Rightarrow 2.6$; $\psi = 3 \Rightarrow 18$; $\psi = 4 \Rightarrow 30$

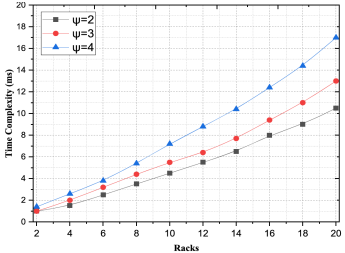
40% improvement over baselines at $\psi = 4$.

Key Values (8 racks)

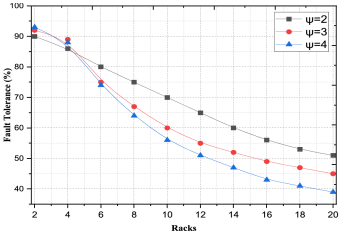
$\psi = 2 \Rightarrow 69\%$; $\psi = 3 \Rightarrow 75\%$; $\psi = 4 \Rightarrow 77\%$

Higher RF = more reliable packet delivery.

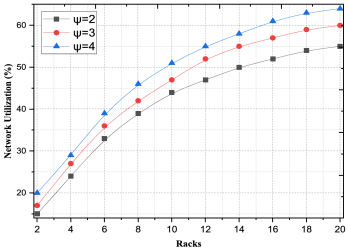
Result 3: Time Complexity, Fault Tolerance & Network Utilization



(c) Time Complexity 12 racks:
$\psi = 2 \rightarrow 5.5$; $\psi = 3 \rightarrow 6.4$;
$\psi = 4 \rightarrow 8.8$

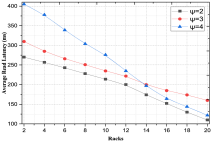


(d) Fault Tolerance 6 racks:
$\psi = 2 \rightarrow 80\%$; $\psi = 3 \rightarrow 75\%$;
$\psi = 4 \rightarrow 74\%$
Higher RF adds sync overhead.

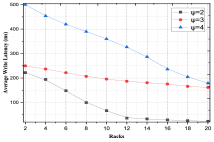


(e) Network Utilization 10 racks:
$\psi = 2 \rightarrow 44\%$; $\psi = 3 \rightarrow 47\%$;
$\psi = 4 \rightarrow 51\%$

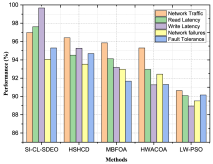
Result 4: Read/Write Latency & Comparative Performance



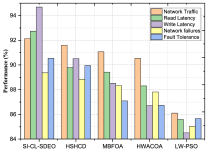
Read Lat.
8 racks:
$\psi = 2 \rightarrow 228$
$\psi = 3 \rightarrow 251$
$\psi = 4 \rightarrow 304$ ms



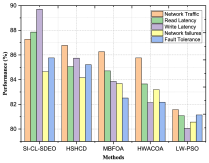
Write Lat.
6 racks:
$\psi = 2 \rightarrow 148$
$\psi = 3 \rightarrow 222$
$\psi = 4 \rightarrow 420$ ms



Low Load
Net: 96.98%
RLat: 97.62%
WLat: 99.67%



Med. Load
Net: 92.13%
RLat: 92.74%
WLat: 94.69%



High Load
Net: 87.28%
RLat: 87.86%
WLat: 89.70%

Key Takeaway

SI-CL-SDEO **consistently leads** HSHCD, MBFOA, HWACOA, and LW-PSO across all load levels on all five performance metrics.

Result 5: Execution Time Comparison

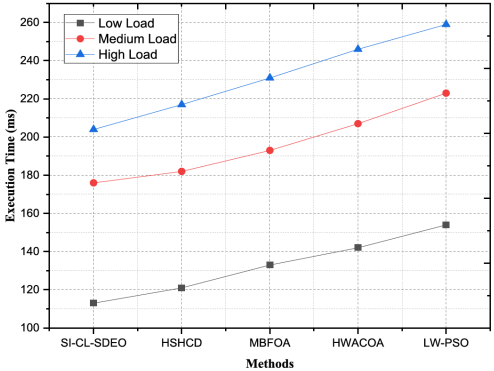


Fig. 10 Comparative Execution Time (ms)

Algorithm	Low	Med	High
SI-CL-SDEO	**113**	**176**	**204**
HSHCD	121	182	217
MBFOA	139	193	238
HWACOA	142	207	246
LW-PSO	154	223	259

Table: Execution times in milliseconds.

Why SI-CL-SDEO is Fastest

- $O(nT)$ vs. $O(n^2T)$ for MBFOA & LW-PSO.
- Leader-follower hierarchy reduces redundant computation.
- Lightweight DE operators ensure rapid convergence.
- Early elimination of low-quality solutions.

Conclusion

Summary

SI-CL-SDEO is a **holistic, scalable, dynamically adaptive** framework for HDFS replication — the first to achieve $O(n \cdot T)$ complexity while optimizing all key metrics simultaneously.

Key Results:

- **Data Availability:** Peak 30 at $\psi = 4$ — **40%** $\uparrow$ over baselines.
- **Execution Time:** 113/176/204 ms — **20%** $\downarrow$ vs. next-best.
- **Fault Tolerance: 5–10%** $\uparrow$ over existing methods.
- **Network Traffic: 96.98%** efficiency (low load).
- **Read/Write Latency:** 160/162 ms (medium load, high RF).

Why It Works

- **Hybrid SSO+DE** resolves exploration-exploitation trade-off.
- **Linear** $O(nT)$ avoids quadratic bottlenecks.
- **Dynamic RF** adapts to real-time cluster state.
- **Single framework** optimizes availability, latency, fault tolerance, and network utilization together.

Future Work

Limitation 1: Static Hyperparameters

W_1=0.7, W_2=0.3, F, CR are pre-tuned. Limits adaptability under volatile workloads.

Proposed — ML-Driven Tuning:

- Deep neural networks or Gaussian Process Regression trained on cluster telemetry.
- Predict optimal RF dynamically under shifting workloads.
- Graphical techniques to visualize data dependencies.

Limitation 2: Homogeneous Cluster

Simulation used identical hardware. Real-world clusters are heterogeneous.

Proposed Extensions:

- Heterogeneous node profiling in fitness function (favour SSD nodes for latency-critical data).
- Validate on edge-to-cloud and erratic network topologies.
- Integrate zero-trust security for data replication.
- Test under extreme simultaneous multi-node failures.

References

1. D. Dhinakaran et al., "SI-CL-SDEO algorithm for improving HDFS performance," *Results Eng.*, vol. 26, p. 104773, 2025.
2. N. J. Kumar, D. Dhinakaran et al., "Swarm intelligence with chaotic leader: HDFS optimization," *Concurr. Comput.*, vol. 36, no. 17, 2024.
3. T. Ma, F. Tian, B. Dong, "Ordinal optimization-based performance model for HDFS," *IEEE Access*, vol. 8, pp. 889–899, 2020.
4. Y. Guan, Z. Ma, L. Li, "HDFS optimization via hierarchical hot/cold data," *Procedia CIRP*, vol. 83, pp. 415–418, 2019.
5. M. Malik, Suman, "LW-PSO for load balancing on cloud," *Wirel. Pers. Commun.*, vol. 125, no. 2, 2022.
6. C. Chandrashekar et al., "HWACOA scheduler for cloud task scheduling," *Appl. Sci.*, vol. 13, no. 6, 2023.
7. P. Krishnadoss et al., "CCSA: cuckoo crow search for cloud scheduling," *Int. J. Intell. Eng. Syst.*, vol. 14, no. 4, 2021.
8. N. Bacanin et al., "Modified firefly for cloud-edge workflow scheduling," *Neural Comput. Appl.*, vol. 34, no. 11, 2022.
9. B. Jin, X. Xu, "Gaussian process regression for wholesale price forecasting," *Neural Comput. Appl.*, vol. 36, pp. 8693–8710, 2024.
10. J. K. Konjaang, L. Xu, "MOWOS: multi-objective workflow optimization," *J. Cloud Comput.*, vol. 10, no. 1, 2021.

Full list (37 refs): D. Dhinakaran et al., Results in Engineering, vol. 26, 2025.